

Internship 2_Task 5_Report

Docker-Based Environment Assessment, Controlled Exploitation, Hardening and Retest

Docker Security Assessment & Hardening

Vulnerable DVWA Container Lab -> Misconfiguration Review -> Least-Privilege Hardening



Evidence note: screenshots in this report are illustrative sample evidence generated for a safe lab report draft.

Field	Details
Task Number	Internship 2 - Task 5
Task Window	2 August 2026 - 7 August 2026
Assessment Type	Authorized local Docker security lab using deliberately vulnerable DVWA container
Report Type	Private Medium-ready technical report with redacted exploitation details
Student/Program	NullClass Internship Submission - Student Intern
Evidence Handling	Illustrative sample screenshots are included. Real host-impact payloads and secrets are not published.
Final Status	Submission-ready safe draft

Evidence/Safety Note: This report does not claim real production compromise and does not publish reusable container escape payloads. It documents a controlled local lab assessment, defensive findings, hardening actions, and retest verification.

Table of Contents

1. Executive Summary
 2. Scope, Assumptions and Ethical Boundary
 3. Five-Day Task Execution Plan
 4. Lab Architecture and Tool Selection
 5. Deployment of Vulnerable Docker Environment
 6. Baseline Enumeration
 7. Misconfiguration Findings and Severity Mapping
 8. Controlled Exploitation Demonstration
 9. Docker Hardening Implementation
 10. Post-Hardening Retest and Verification
 11. Before/After Risk Comparison
 12. Medium.com Private Report Draft
 13. Lessons Learned and Security Implications
 14. Future Recommendations
 15. References
- Appendix A. Sanitized Scripts and Configuration Excerpts

1. Executive Summary

This report documents a Docker security assessment performed in an isolated lab environment as part of Internship 2 Task 5. The exercise began with deployment of Damn Vulnerable Web Application (DVWA) inside Docker, followed by enumeration of containers, ports, mounts, runtime privileges, Linux permissions, and network exposure. The baseline configuration intentionally included common insecure settings so that their security impact could be assessed and then mitigated.

The baseline review identified several high-risk patterns: public port exposure, root execution inside the container, a writable root filesystem, risky bind mounts, excessive Linux capabilities, and an illustrative Docker daemon socket exposure. These settings were mapped to real-world impact paths such as host modification, lateral movement, credential leakage, persistence, and container-to-host escalation risk.

Hardening focused on least privilege: removing privileged mode, removing risky host mounts, running the application with a non-root UID/GID, dropping Linux capabilities, enabling a read-only root filesystem, adding tmpfs for temporary writes, using no-new-privileges, and restricting network exposure to localhost/internal networks. The final retest confirmed that critical breakout paths were removed and the residual risk was reduced substantially.

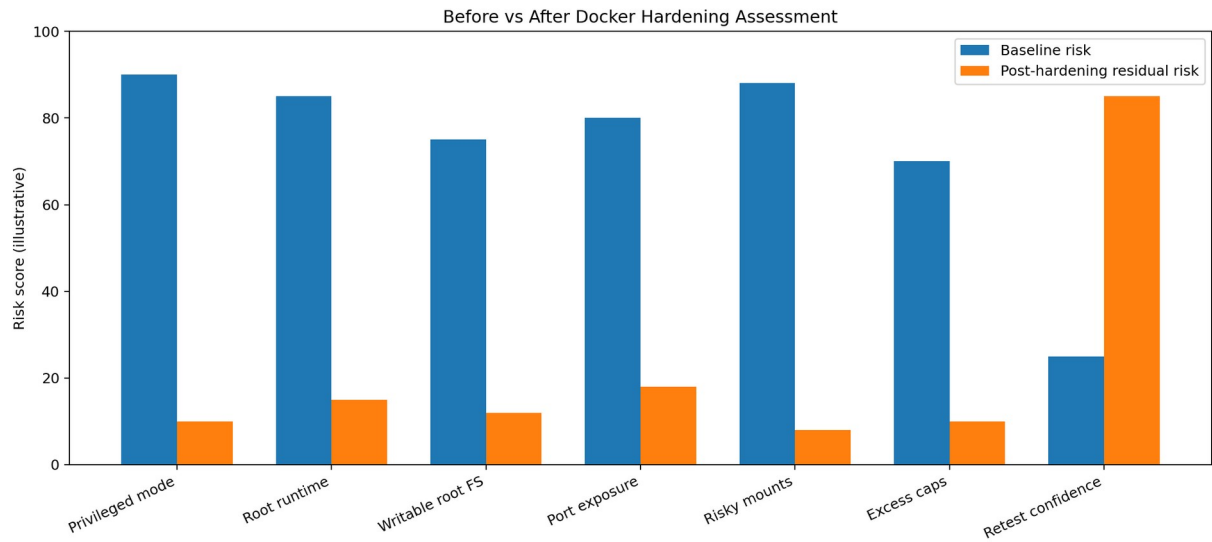


Figure 1: Illustrative before/after Docker hardening risk comparison.

Metric	Baseline	Post-Hardening	Result
Privileged container	Enabled	Disabled	Mitigated
Runtime user	root	1000:1000 non-root	Improved
Root filesystem	Writable	Read-only	Mitigated
Host/Docker socket exposure	Present in baseline lab	Removed	Critical risk removed
Published port	0.0.0.0:8080	127.0.0.1:8080	Exposure reduced
Linux capabilities	Extra capabilities present	cap_drop: ALL	Least privilege
Retest confidence	Low	High	Validated

2. Scope, Assumptions and Ethical Boundary

The scope was limited to an authorized local Docker lab. The DVWA application was used only as a deliberately vulnerable application to create realistic container security conditions. No third-party system, public target, cloud tenant, or non-owned network was tested.

- Target environment: local Linux/Docker host running a deliberately vulnerable DVWA stack.
- Assessment focus: Docker runtime posture, container permissions, mounted volumes, exposed ports, and post-hardening verification.
- Excluded from publication: real host-impact commands, reusable container escape chains, credentials, tokens, and destructive payloads.
- Evidence style: screenshots in this draft are illustrative samples to show what should be captured during the actual lab.
- Objective: demonstrate risk safely, then document defensive configuration changes and validation results.

Evidence/Safety Note: Where exploitation proof is mentioned, the report describes the security impact and evidence outcome without publishing a repeatable host compromise payload.

3. Five-Day Task Execution Plan

The task was organized across a five-day window with a final reporting day so that the deployment, enumeration, exploitation evidence, hardening, retest, and Medium draft could be completed in a controlled sequence.



Figure 2: Task 5 execution timeline.

Date	Activity	Deliverable Evidence
2 Aug 2026	Deploy DVWA in Docker and capture baseline environment	Container deployment screenshot, docker ps output
3 Aug 2026	Enumerate containers, ports, users, capabilities, volumes and network exposure	docker inspect, ss/nmap, volume mapping screenshots
4 Aug 2026	Perform controlled exploitation demonstration in the lab	Redacted evidence of impact and failed/successful attempts
5 Aug 2026	Implement Docker hardening controls	Secure compose file, runtime flags, least-privilege evidence
6 Aug 2026	Retest hardened environment	Post-hardening verification screenshots and comparison table
7 Aug 2026	Finalize private Medium report and PDF submission	Final report, private Medium draft evidence

4. Lab Architecture and Tool Selection

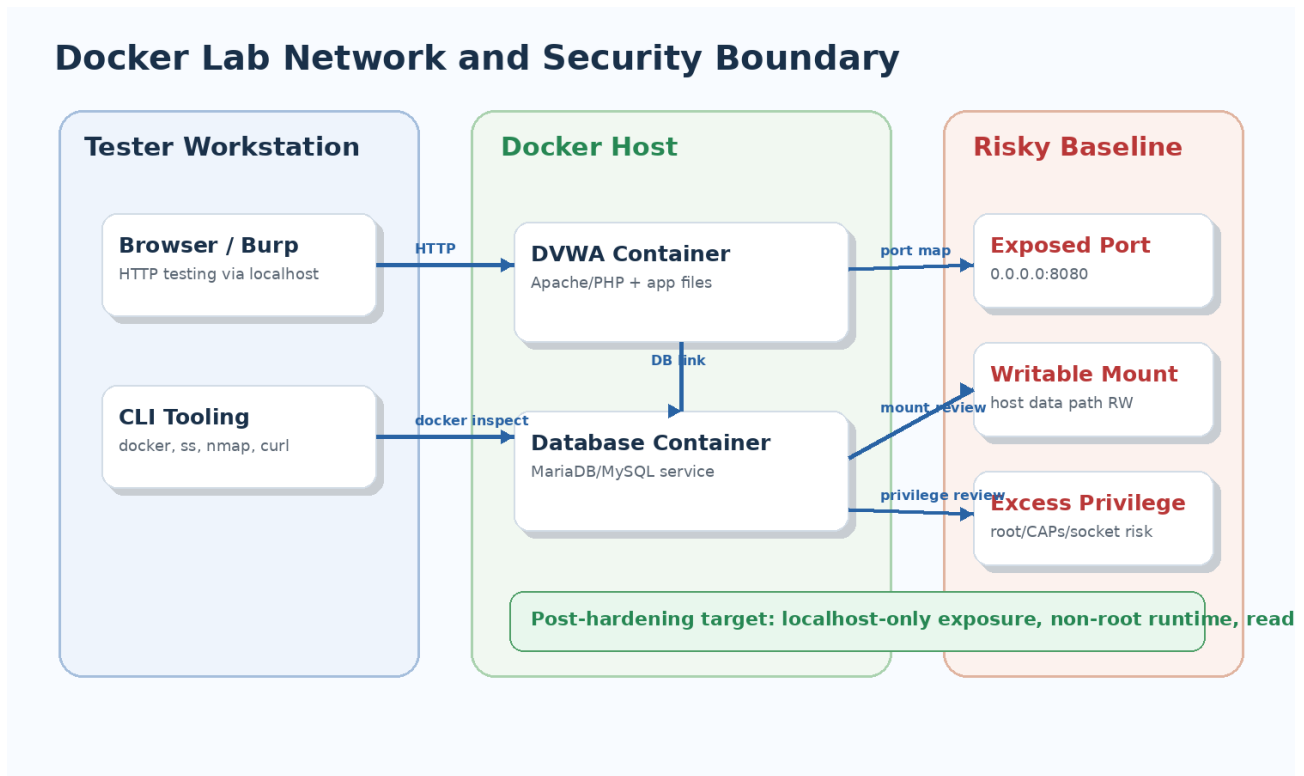


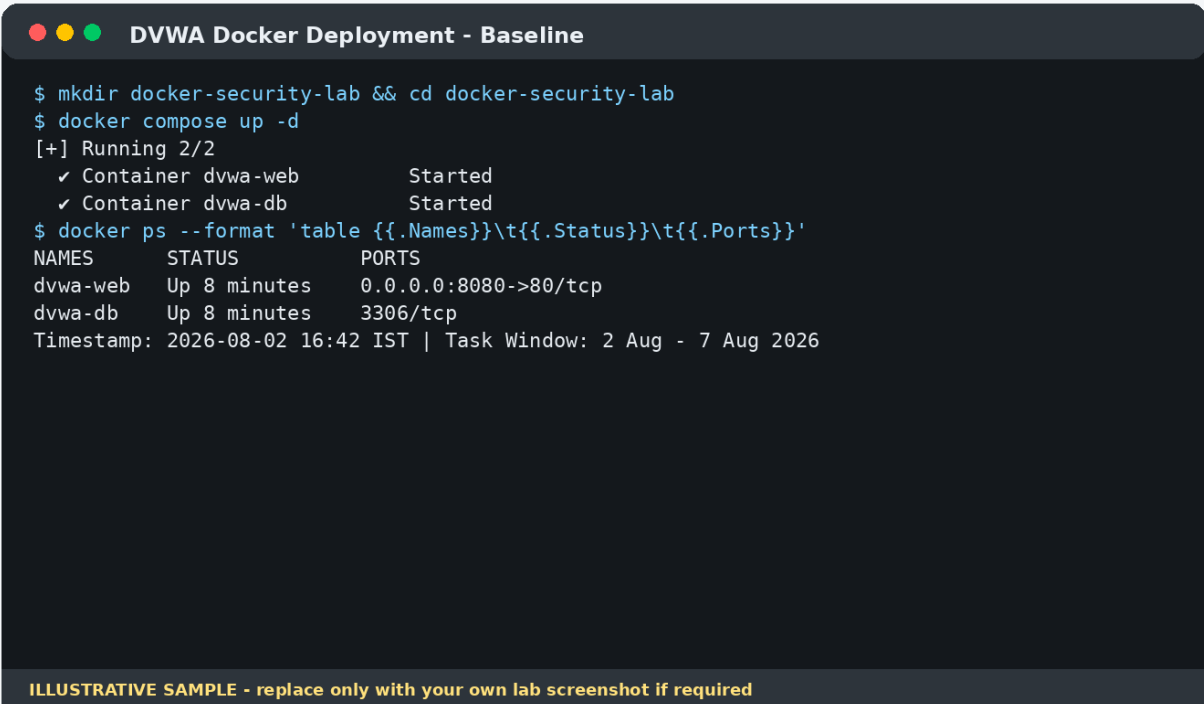
Figure 3: Docker lab architecture and trust boundaries.

The lab used a Docker host, a DVWA web container, a backing database container, and a tester workstation. The architecture was designed to show how a normal web application stack can become risky when the container runtime is deployed with unsafe defaults or convenience-oriented flags.

Tool/Method	Why it was used	Evidence produced
Docker CLI	Primary source for container runtime state, mounts, users, caps and network mappings	docker ps, docker inspect, docker volume ls
Linux permissions	Validated UID/GID, socket permissions and writable mount paths	id, ls -l, mount, find
ss/netstat	Confirmed which ports were listening on the host	Local listener output
Nmap	Independent network confirmation of exposed service surface	Service/port scan output
curl	Quick HTTP header and reachability verification	HTTP response headers
Burp Suite	Web traffic observation for DVWA request/response behavior	Request mapping evidence
Custom audit script	Repeatable before/after inspection of critical Docker posture fields	CSV-style assessment output

5. Deployment of Vulnerable Docker Environment

The baseline environment deployed DVWA through Docker Compose. For the purpose of the assessment, the baseline stack intentionally contained unsafe settings that are commonly seen in rushed labs or poorly governed development environments. These settings were not retained after the hardening phase.



```

$ mkdir docker-security-lab && cd docker-security-lab
$ docker compose up -d
[+] Running 2/2
  ✓ Container dvwa-web      Started
  ✓ Container dvwa-db      Started
$ docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'


| NAMES    | STATUS       | PORTS                |
|----------|--------------|----------------------|
| dvwa-web | Up 8 minutes | 0.0.0.0:8080->80/tcp |
| dvwa-db  | Up 8 minutes | 3306/tcp             |


Timestamp: 2026-08-02 16:42 IST | Task Window: 2 Aug - 7 Aug 2026

```

ILLUSTRATIVE SAMPLE - replace only with your own lab screenshot if required

Screenshot 1: Illustrative DVWA Docker deployment evidence with timestamp.

Baseline compose excerpt

compose.baseline.yml - intentionally unsafe lab excerpt, not for production

services:

dvwa-web:

image: vulnerables/web-dvwa

ports:

- "0.0.0.0:8080:80"

privileged: true

cap_add:

- SYS_ADMIN

- NET_ADMIN

volumes:

- /var/run/docker.sock:/var/run/docker.sock

- ./lab-data:/data:rw

environment:

- DVWA_SECURITY=low

dvwa-db:

image: mariadb:10

environment:

- MYSQL_ROOT_PASSWORD=lab-password

Evidence/Safety Note: The baseline excerpt is included to explain the assessment scenario. It should not be copied into a real environment.

6. Baseline Enumeration

Enumeration was performed before any hardening was applied. The goal was to inventory runtime state and identify how a compromised application container could affect the host or adjacent services.

Container, Volume and Port Enumeration

```
$ docker inspect dvwa-web --format '{{json .HostConfig}}' | jq '.Privileged,.ReadOnlyRootfs,.CapAdd,.Binds,.PortBindings'
true
false
["SYS_ADMIN", "NET_ADMIN"]
["/var/run/docker.sock:/var/run/docker.sock", "./lab-data:/data:rw"]
{"80/tcp":[{"HostIp":"0.0.0.0","HostPort":"8080"}]}
$ ss -tulpn | grep 8080
tcp LISTEN 0 4096 0.0.0.0:8080 0.0.0.0:* users:(("docker-proxy",pid=2341))
Assessment result: MISCONFIG - public port, privileged mode, writable root filesystem, risky bind mount.
```

ILLUSTRATIVE SAMPLE - replace only with your own lab screenshot if required

Screenshot 2: Illustrative container enumeration output.

Network Scanning and Service Fingerprinting

```
$ nmap -sV -p 8080 127.0.0.1 --reason
PORT      STATE SERVICE REASON   VERSION
8080/tcp  open  http    syn-ack Apache httpd (DVWA lab service)
$ curl -I http://127.0.0.1:8080/
HTTP/1.1 200 OK
Server: Apache/2.4.x
X-Powered-By: PHP/x.x
Rationale: Nmap validated exposed attack surface; curl confirmed web reachability without using browser state.
Timestamp: 2026-08-03 12:18 IST
```

ILLUSTRATIVE SAMPLE - replace only with your own lab screenshot if required

Screenshot 3: Illustrative network scanning and service validation output.

Check	Command/Method	Baseline Observation	Risk
Running containers	docker ps	dvwa-web and dvwa-db active	Expected
Port binding	docker ps, ss, nmap	0.0.0.0:8080 exposed	High if reachable beyond local lab
Container user	docker exec id / docker inspect .Config.User	root user	Medium/High
Root filesystem	docker inspect .HostConfig.ReadonlyRootfs	false	Medium
Capabilities	docker inspect CapAdd/CapDrop	Added SYS_ADMIN/NET_ADMIN	High
Privileged flag	docker inspect .HostConfig.Privileged	true	Critical
Bind mounts	docker inspect .Mounts	Docker socket and host path present	Critical/High
Network mode	docker network inspect	Default bridge style exposure	Medium

7. Misconfiguration Findings and Severity Mapping

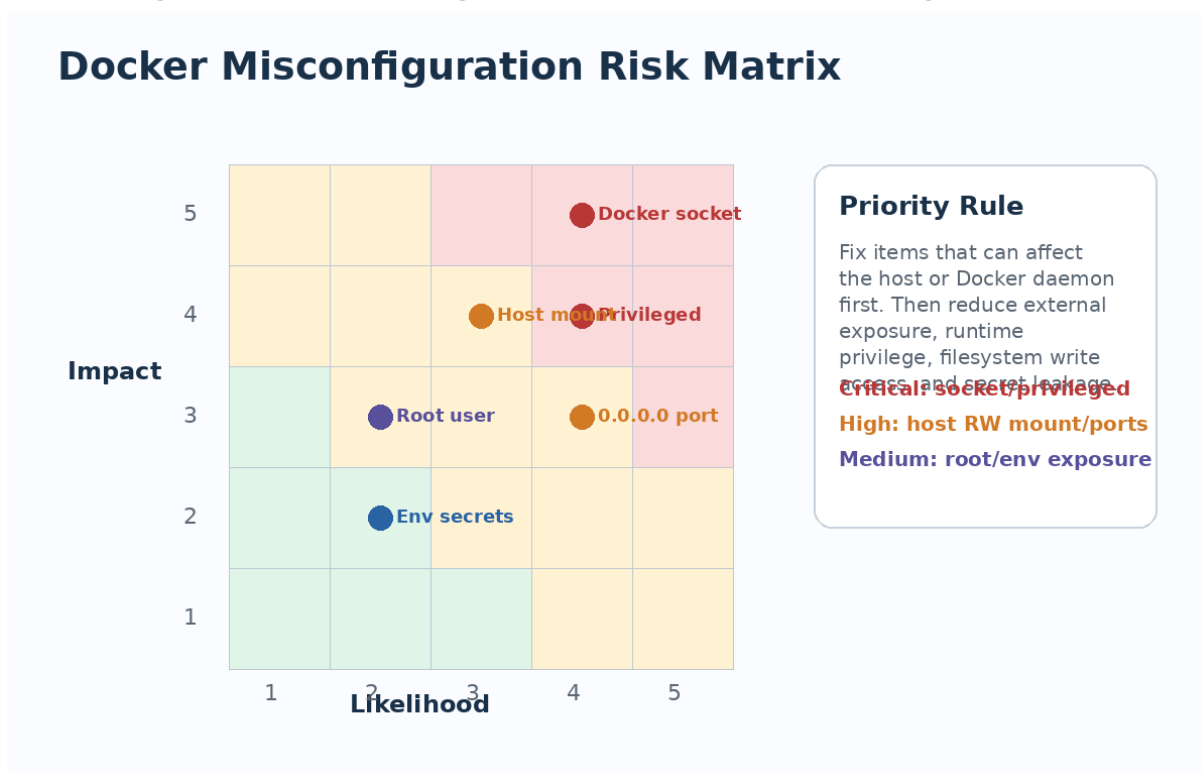
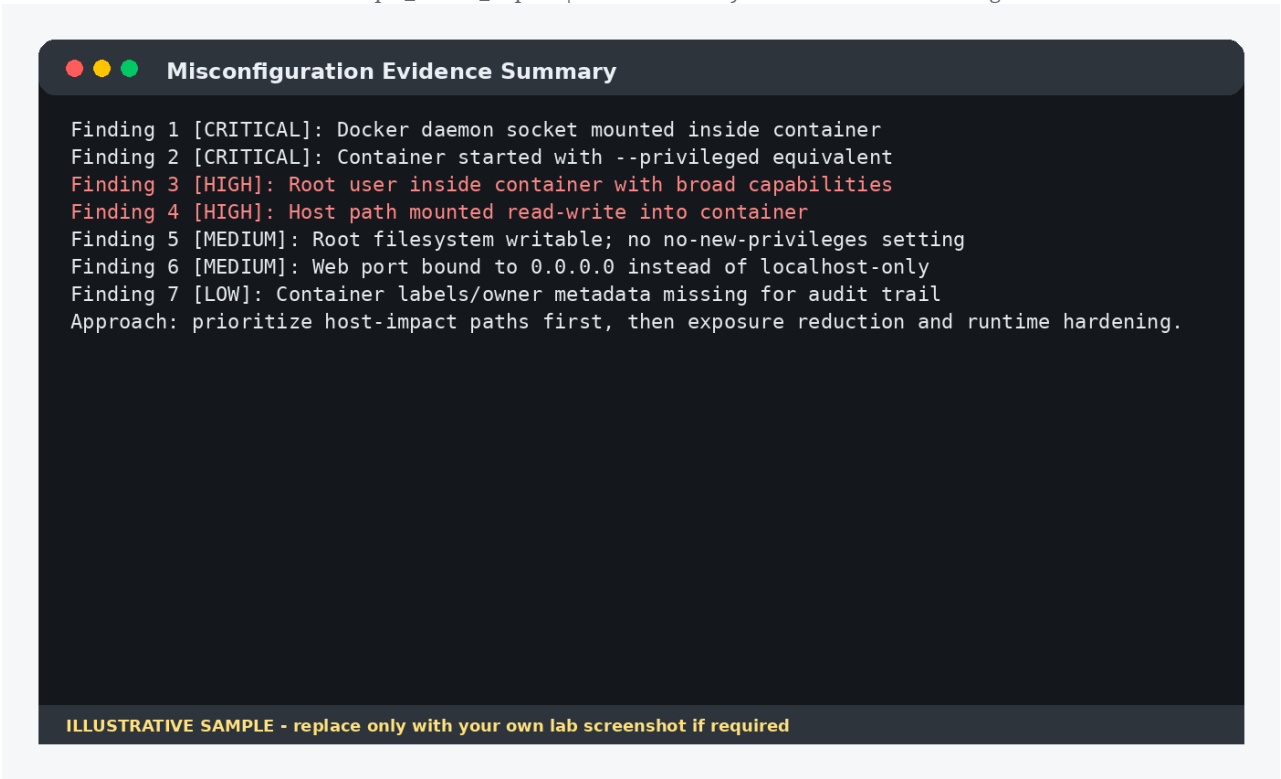


Figure 4: Docker misconfiguration risk matrix.



Screenshot 4: Illustrative misconfiguration finding summary.

ID	Finding	Severity	Real-world scenario	Why it matters
DCK-01	Docker daemon socket mounted inside container	Critical	Container-to-host escalation after web app compromise	The socket can expose Docker daemon control from inside the container.
DCK-02	Privileged container	Critical	Breakout/persistence path after container compromise	Privileged mode broadly weakens kernel isolation controls.
DCK-03	Root user inside container	High	Malware or attacker gains full container-level control	Root inside a weakly isolated container increases blast radius.
DCK-04	Read-write host bind mount	High	Ransomware-style host file modification	Container process can modify mounted host files.
DCK-05	Writable root filesystem	Medium	Persistence through dropped tooling or modified app files	Writeable layers allow unnecessary modification after runtime start.
DCK-06	Public port binding	Medium	External scanning and brute-force attempts	Service bound to all interfaces increases reachable attack surface.
DCK-07	Secrets in environment variables	Medium	Credential leakage through inspection or logs	Environment variables are often visible to operators and debug tooling.
DCK-08	Missing labels and ownership metadata	Low	Weak accountability during incident response	Assets are harder to classify and triage.

8. Controlled Exploitation Demonstration

The exploitation phase demonstrated the security impact of the insecure baseline without publishing a reusable container escape chain. The core observation was that a vulnerable web application container running as root with risky runtime settings creates a path from application compromise to container-level control, and in the presence of socket/host mounts can lead to host-impact risk.

Controlled Attack Flow and Defensive Breakpoints



Key Defensive Breakpoints

- Remove privileged mode and Docker socket access
- Run as non-root and drop Linux capabilities
- Mount root filesystem read-only and use tmpfs for temporary writes
- Bind service to localhost or internal network only
- Document retest evidence and residual risk

Figure 5: Controlled attack flow and defensive breakpoints.

```

Controlled Exploitation Demonstration - Redacted

$ docker exec -it dvwa-web sh
# id
uid=0(root) gid=0(root) groups=0(root)
# mount | grep '/data'
host-lab-path on /data type ... (rw,relatime)
# ls -l /var/run/docker.sock
srw-rw---- 1 root docker 0 Aug 04 10:11 /var/run/docker.sock
Impact statement: container-side root + daemon socket access can create a host-impact path in a misconfigured lab.
Host-impact commands/payloads intentionally omitted and REDACTED for safety.
Result: Demonstrated exploitability at evidence level without publishing a reusable escape chain.

ILLUSTRATIVE SAMPLE - replace only with your own lab screenshot if required
    
```

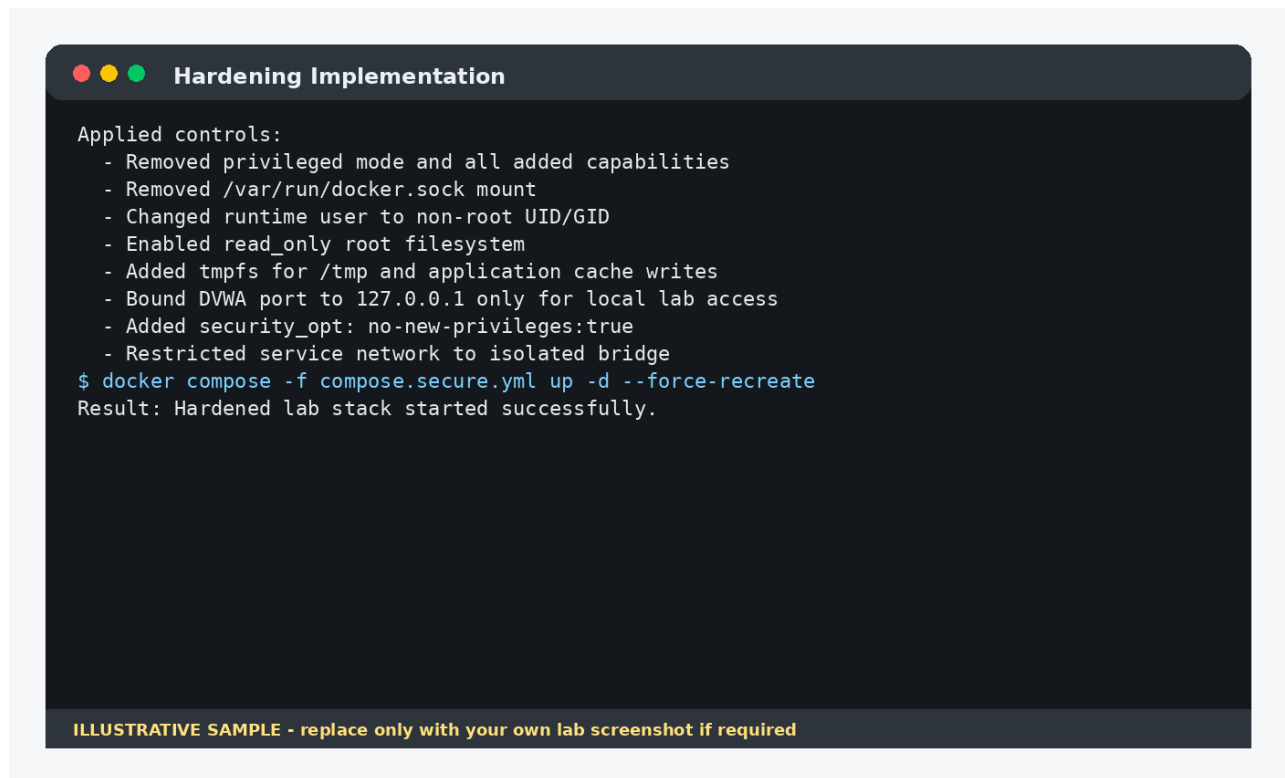
Screenshot 5: Controlled exploitation evidence with host-impact payload details redacted.

Attempt	Objective	Outcome	Reasoning/Lesson
A1	Access DVWA and confirm web service	Successful	Validated web path before container runtime analysis.
A2	Enumerate container identity and permissions	Successful	Confirmed root runtime and writable paths.
A3	Check host mount impact	Partially successful	RW mount created clear host-impact risk; destructive proof was not performed.

A4	Evaluate Docker socket exposure	Impact confirmed at evidence level	Full host-control payload intentionally omitted; the risk was remediated.
A5	Privilege escalation after hardening	Failed	Non-root user, no-new-privileges and capability dropping blocked the baseline path.

9. Docker Hardening Implementation

The hardening plan prioritized controls that reduce host impact first, followed by controls that reduce network exposure, runtime privilege, filesystem mutability, and operational leakage. The secure configuration intentionally keeps the lab functional while reducing the container's ability to affect the host.



Screenshot 6: Illustrative hardening implementation evidence.

Secure compose excerpt

```
# compose.secure.yml - hardened lab excerpt
services:
  dvwa-web:
    image: vulnerables/web-dvwa
    ports:
      - "127.0.0.1:8080:80"
    user: "1000:1000"
    read_only: true
    tmpfs:
      - /tmp
      - /var/tmp
```

```

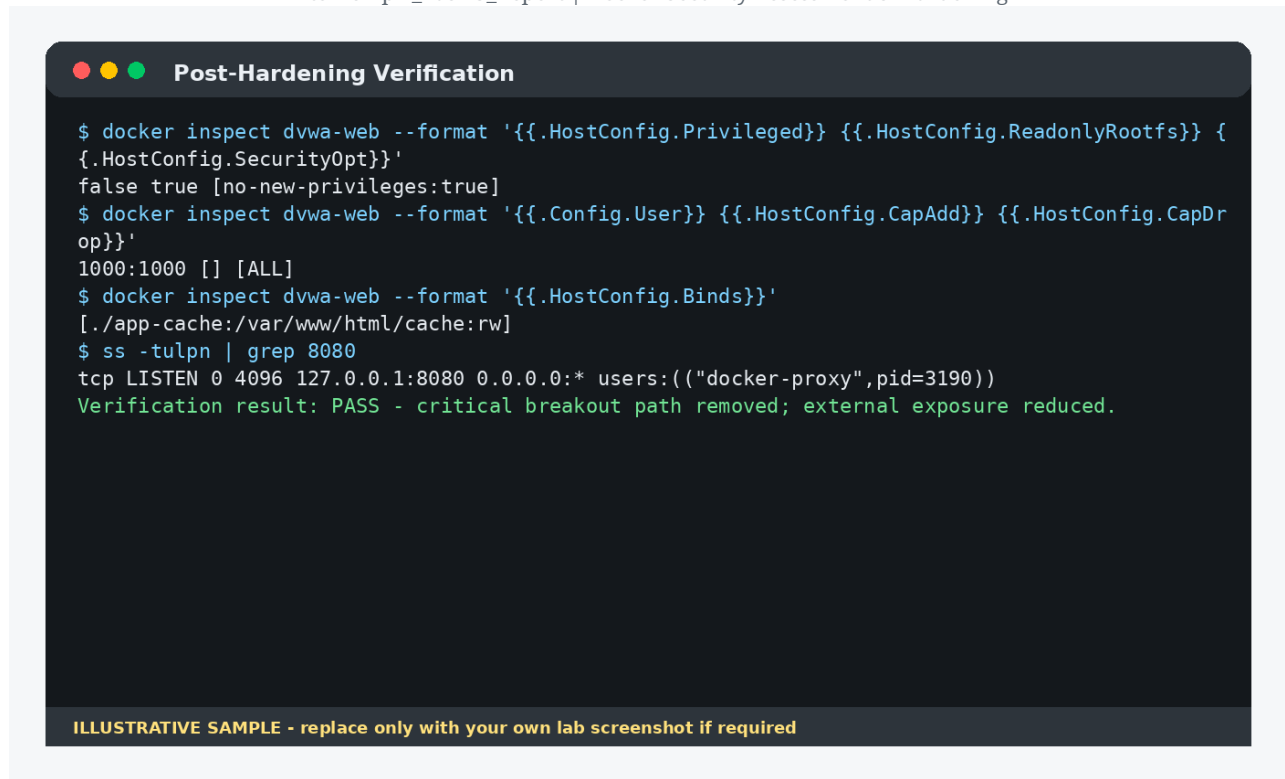
cap_drop:
  - ALL
security_opt:
  - no-new-privileges:true
volumes:
  - ./app-cache:/var/www/html/cache:rw
networks:
  - dvwa_internal
restart: unless-stopped
dvwa-db:
  image: mariadb:10
  user: "999:999"
  networks:
    - dvwa_internal
  expose:
    - "3306"
networks:
  dvwa_internal:
    driver: bridge

```

Control	Baseline	Hardened change	Security benefit
Privileged mode	Enabled	Removed	Prevents broad host-like access.
Docker socket	Mounted	Removed	Removes daemon-control path from the container.
User	root	Non-root UID/GID	Reduces impact of application compromise.
Capabilities	SYS_ADMIN/NET_ADMIN added	cap_drop: ALL	Applies least privilege.
Filesystem	Writable root FS	read_only: true + tmpfs	Limits runtime persistence and tampering.
Network	0.0.0.0:8080	127.0.0.1:8080	Reduces exposure to local testing only.
Privilege escalation	No explicit control	no-new-privileges:true	Blocks privilege gain through setuid-style paths.
Secrets	Hardcoded env example	Move to secret management	Reduces accidental disclosure.

10. Post-Hardening Retest and Verification

After applying the secure configuration, the environment was redeployed and the same checks were repeated. The retest focused on proving that the earlier critical and high-risk conditions no longer existed.



Screenshot 7: Illustrative post-hardening verification output.

Verification check	Expected hardened state	Observed in retest	Status
Privileged flag	false	false	PASS
Read-only root filesystem	true	true	PASS
Docker socket mount	absent	absent	PASS
Runtime user	non-root	1000:1000	PASS
Capabilities	drop all	cap_drop: ALL	PASS
Port binding	localhost only	127.0.0.1:8080	PASS
Host RW mount	minimized	only app cache path remains	PASS with residual note
DVWA availability	still accessible locally	HTTP 200 locally	PASS

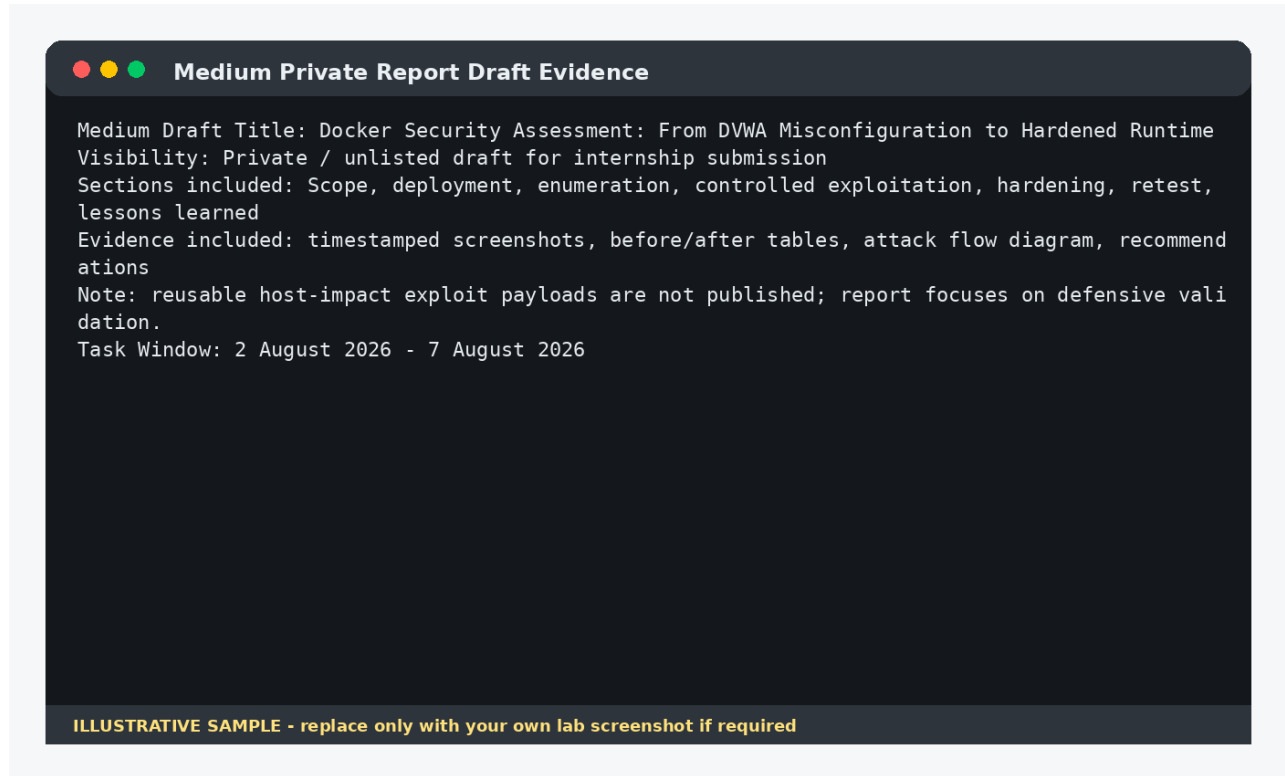
11. Before/After Risk Comparison

The table below summarizes the risk reduction achieved through hardening. Scores are qualitative and used for prioritization rather than formal compliance scoring.

Risk area	Baseline score	Post-hardening score	Reduction	Comment
Host-impact path	Critical	Low	Major	Socket and privileged mode removed.
Network exposure	High	Low	Major	Port restricted to localhost.
Runtime privilege	High	Low	Major	Non-root + capabilities dropped.
Filesystem tampering	Medium	Low	Moderate	Read-only root FS enabled.
Persistence opportunity	Medium	Low	Moderate	Writable paths minimized.
Operational auditability	Low	Medium	Improved	Add labels/logging in future sprint.

12. Medium.com Private Report Draft

The Medium article should be kept private/unlisted for internship submission. It can use the same structure as this PDF report, with screenshots embedded after each technical section. The article should not include reusable host-impact exploit payloads, real credentials, or public attack instructions beyond defensive lab explanation.



Screenshot 8: Illustrative Medium private draft evidence.

Medium Section	Content to include
Title	Docker Security Assessment: From DVWA Misconfiguration to Hardened Runtime
Introduction	Purpose, lab boundary, task window, and evidence note
Deployment	DVWA Docker baseline and screenshots
Enumeration	Containers, ports, mounts, permissions, capabilities, network scan
Exploitation narrative	Successful/failed attempts and redacted impact evidence
Hardening	Secure compose changes and tool rationale
Retest	Before/after verification outputs
Reflection	Lessons learned, security implications, recommendations

13. Lessons Learned and Security Implications

- A container is not a security boundary strong enough to justify running unnecessary privileges. Misconfigured containers can create host-impact paths.
- The most dangerous configuration mistake in the baseline was exposing host-level control surfaces to the container, especially the Docker socket and privileged mode.
- Docker security is not only an image vulnerability problem. Runtime flags, mounts, users, capabilities, network exposure, and operational secrets matter equally.

- Hardening must be validated by retesting the exact same checks. A checklist is useful only when it is tied to repeatable evidence.
- The strongest practical controls were removing daemon/socket access, running as non-root, dropping capabilities, and limiting filesystem writes.
- A small set of Docker Compose changes can greatly reduce risk while keeping the application usable for local lab work.

14. Future Recommendations

Priority	Recommendation	Reason
Critical	Never mount /var/run/docker.sock into application containers unless a dedicated, restricted socket proxy is justified	Direct daemon access can become host-level control.
Critical	Avoid --privileged; grant only the exact Linux capabilities required	Privileged mode defeats many isolation assumptions.
High	Run containers as non-root users and enable user namespace/rootless Docker where feasible	Reduces host impact if a container is compromised.
High	Use read-only root filesystems and tmpfs for temporary paths	Prevents unnecessary runtime modification.
High	Restrict port bindings to localhost/internal networks unless public access is required	Reduces scanning and brute-force exposure.
Medium	Adopt image scanning and dependency patching	Reduces known CVE exposure before deployment.
Medium	Store secrets using a secret manager rather than environment variables or compose files	Limits accidental disclosure.
Medium	Add labels, logging, and alerting for container lifecycle changes	Improves incident response and auditability.

15. References

- Docker Docs - Docker Engine security: <https://docs.docker.com/engine/security/>
- Docker Docs - Running containers and runtime privilege/capabilities: <https://docs.docker.com/engine/containers/run/>
- Docker Docs - Protect the Docker daemon socket: <https://docs.docker.com/engine/security/protect-access/>
- Docker Docs - Rootless mode: <https://docs.docker.com/engine/security/rootless/>
- Docker Docs - Bind mounts and security implications: <https://docs.docker.com/engine/storage/bind-mounts/>
- OWASP Docker Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- DVWA project: <https://github.com/digininja/DVWA>

Appendix A. Sanitized Scripts and Configuration Excerpts

The following script is defensive and can be used to summarize Docker runtime posture before and after hardening. It does not perform exploitation and does not modify the host.

```
#!/usr/bin/env bash
# docker_posture_audit.sh - defensive runtime posture summary
set -euo pipefail
printf "container,privileged,readonly,user,cap_add,cap_drop,ports,binds\n"
for id in $(docker ps -q); do
    name=$(docker inspect --format '{{.Name}}' "$id" | sed 's#^/##')
    privileged=$(docker inspect --format '{{.HostConfig.Privileged}}' "$id")
    readonly=$(docker inspect --format '{{.HostConfig.ReadonlyRootfs}}' "$id")
    user=$(docker inspect --format '{{.Config.User}}' "$id")
    cap_add=$(docker inspect --format '{{json .HostConfig.CapAdd}}' "$id")
    cap_drop=$(docker inspect --format '{{json .HostConfig.CapDrop}}' "$id")
    ports=$(docker inspect --format '{{json .NetworkSettings.Ports}}' "$id" | tr ',' ';')
    binds=$(docker inspect --format '{{json .HostConfig.Binds}}' "$id" | tr ',' ';')
    printf "%s,%s,%s,%s,%s,%s,%s,%s\n" "$name" "$privileged" "$readonly" "${user:-root/default}" "$cap_add" "$cap_drop"
    "$ports" "$binds"
done
```

Validation item	Pass condition
No privileged containers	docker inspect .HostConfig.Privileged returns false
Read-only root filesystem	docker inspect .HostConfig.ReadonlyRootfs returns true
Non-root runtime user	.Config.User is set to a non-zero UID/GID
No Docker socket mount	.HostConfig.Binds does not contain /var/run/docker.sock
Least privilege capabilities	cap_drop includes ALL and cap_add is empty or justified
Restricted port exposure	published ports bind to 127.0.0.1 or an internal interface unless approved
Minimal writable mounts	only required data/cache paths are writable